

Project Proposal: Regional Nectar-Flow Forecast

16-Day Hive-Weight Prediction for the Central-Swiss Pre-Alps

I.BA_MLOPS · HS26

Elias Christen

<https://github.com/Whalesharkking/mlops>

1. Problem statement

The goal is to predict how much weight the monitored bee colonies in the *Nordalpen* region gain or lose each day, for each of the next 16 days. Nordalpen is a Central-Swiss pre-alpine zone covering roughly the cantons of Lucerne, Schwyz, Uri and Ob-/Nidwalden. The forecast is a series of 16 daily values (horizon $h = 1 \dots 16$ days) and is updated once a day, after the nightly scale upload.

This daily weight change is measured in kilograms, after beekeeper actions (feeding, honey removal, adding boxes) are filtered out. A positive value means the colony collects more nectar than it eats, a negative value means it lives off its stores. The value is averaged over the roughly 372 scales in the region, so the forecast describes the regional nectar flow, not a single hive.

This helps beekeepers plan ahead: when to harvest (once little new nectar is expected), when to feed before a treatment, whether colonies might run low on winter stores, and when a nectar gap (“Trachtlucke”) or a strong flow is coming.

Success criterion (metric: MAE, the mean absolute error in kg/day): the model should beat a persistence baseline (tomorrow is like today) by at least 10 %, and reach or beat a seasonal-climatology baseline (the typical net change for that day of the year). The biggest gains are expected in the first week, where the weather forecast is still reliable. It is tested on unseen data from 2025 to today.

2. Originality & motivation

I am an active beekeeper, so I know the problem first-hand and can judge whether the output is useful in the apiary.

I have not come across a nectar-flow forecast built from a live network of hive scales, either in earlier course projects or elsewhere. What makes it different is the data: a Swiss citizen-science network of hundreds of connected scales, combined with weather forecasts, used to predict a biological signal (nectar income) that depends on the weather but cannot be read directly from it.

The real question is not only whether the model beats a rolling average, but whether a 16-day weather forecast lets it beat what a beekeeper already knows from experience: the seasonal pattern that the climatology baseline captures.

3. Data source & features

There are two live sources, both public APIs, with no scraping. The hive data comes from the HiveWatch / BienenSchweiz Waagvölker network. Its endpoint `region/{id}/averages` returns the intervention-filtered `ext_weight` series (the regional average hive weight), going back to 2019 at roughly hourly resolution and updated every night. Access needs only a fixed public request header, with no key and no login. This gives about seven years of history today, and it grows by one day every day. The weather comes from Open-Meteo, whose free API gives both a 16-day forecast and a historical archive back to 2019 for the centre of the region, again with no key. If the feed ever changes, a fallback source (another open hive-scale network, or a climatology-only model) is kept ready.

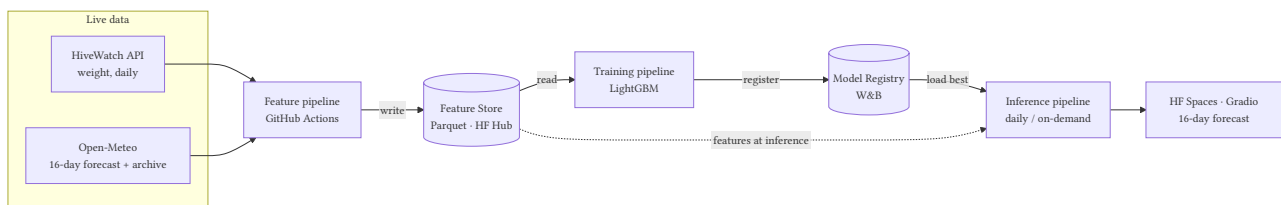
The label (what the model predicts) is the hive's net weight change on a day, in kilograms, and there is one value for each of the next 16 days. It is computed from the weight series alone: keep one weight per day and subtract consecutive days. Because it never uses the weather features, the model cannot read the answer off a feature and must learn the real link between weather and nectar.

The features come in three groups. First, the weather forecast for each target day (max and min temperature, total precipitation, shortwave radiation, and maximum wind), always taken from the forecast available at t_0 and never from the weather that actually happened. Second, the recent weight momentum: the net weight change over the last 1, 3, 7 and 14 days. Third, calendar features (day of the year as sine and cosine, and the month) that encode the season. All rolling windows look only backwards. Because the scale feed is not exactly 24 points per day, the series is placed on a fixed daily grid before any lag is computed, so a "1-day" lag is always a real day, not just a number of rows.

For evaluation, the model is trained on the full 2019 to 2024 history (six seasons) and tested on 2025 to today, kept in time order, with a 16-day gap so the training and test label windows do not overlap. This is a regression task, so there is no rare class to handle.

4. System design

The system follows the feature, training and inference (FTI) split shown below.



4.1. Core

The feature pipeline runs once a day. It fetches the new HiveWatch weight values and the Open-Meteo forecast, computes the features, and adds them to the feature store. A separate backfill run loads the full 2019-to-today history one time. The training pipeline retrains the model from scratch on a fixed schedule (for example weekly), so it always learns from the latest data. Each run trains a gradient-boosted-tree model (LightGBM) that predicts all 16 horizons, compares it against the persistence and climatology baselines, and registers the best version. The inference pipeline loads the latest registered model and shows the 16-day forecast next to the two baselines, either on a daily schedule or on demand from the UI.

4.2. Tech stack

HuggingFace Hub keeps the versioned daily features as Parquet (a “poor man’s feature store”), read by both the training and inference pipeline, free and in the same ecosystem as the UI

Weights & Biases records every training run with its settings and metrics and keeps the versioned models in its model registry, a free cloud with no server to run

GitHub Actions runs the daily feature pipeline and the weekly retraining on a schedule, plus a manual backfill trigger

HuggingFace Spaces (Gradio) serves the 16-day forecast next to the baselines and shows the current model version, a free one-push cloud deployment (the required cloud deploy)

Docker with pinned dependencies reproducible runs on the local machine, in CI and on the Space

Only two secrets are needed (the HuggingFace and W&B tokens), kept in GitHub Secrets. The data sources themselves need none. The GitHub repository is public.

4.3. Optional

Marked optional, only if time allows: mirroring the deployment to GCP Cloud Run / GCS with the student credits, a data-drift check on the weight and weather distributions, per-colony forecasts, and a nectar-gap classifier.